

DATA INTEGRITY

Scoring Data-Contract Audit

Does the canonical-date failure class - data written in a format the scoring model cannot read, silently dropped to a default - exist in other scoring systems?

Question	Is the date-bug disease (silent format misread of scoring inputs) systemic across all scoring models?
Scope	Trend Gradient, Market/Risk Gradient, AI-Grade + AI-taxonomy, Accuracy Ledger + N + Dark Matter, and the engine's format/contract layer
Method	26 AI agents: per-subsystem input-contract mapping + 5 failure-mode lens scans + adversarial verification against live data + synthesis
Result	5 subsystems mapped; 42 candidate silent-misreads; 15 high-severity verified; 9 confirmed real
Verdict	CONFIRMED - the failure class is systemic. One enforced format contract exists (dates only); every other scoring input relies on conventions that mask missing data as a real-looking 0.
Date	2026-06-23

Why this matters (the founder's framing). If a scoring input is in a format the consumer cannot read - wrong type, wrong unit, wrong key, unparsed JSON, or simply absent - it is silently coerced to a default (usually 0) and enters the weighted score as if it were real data. The grade and the gap then look complete but are computed on missing inputs. Accurate grading and gap analysis are impossible until every scoring input is guaranteed to be read in the format it was written.

Executive Summary

FINDING: The canonical-date bug was one instance of a pervasive data-contract failure class. It is present, in confirmed form, across every scoring subsystem.

There is exactly ONE machine-enforced per-field format contract in the engine (`ingestion_gate.DATE_SEMANTIC`) and it covers dates only. Every other scoring input depends on prose docstrings plus the defensive idiom `s.get(key, 0)` or `0`, which converts BOTH a real zero AND a missing/unparseable value into the same 0 - and that 0 enters the weighted sum indistinguishably.

The audit mapped five scoring subsystems and ran five failure-mode lenses across the live scoring code, surfacing 42 candidate silent-misreads; 15 high-severity cases were adversarially verified against live engine data and 9 were confirmed real.

The two most material are HIGH severity. First, the AI-Grade path scores topics with the LEGACY seven-component weight vector that still INCLUDES N (the on-platform demand signal) - so a graded topic is both on a different scale than the engine AND has the exact N-into-Gradient circularity the project's integrity standard forbids re-introduced through the grade door.

Second, the market Positioning Concentration component silently collapses to a flat, non-informative baseline whenever its input is missing, because missing-is-masked-as-zero on that path. Around these sit a cluster of the same disease: KEY DRIFT (`engagement_asymmetry` written under one key and read under another on the scoring path; `nowtrendin_score` vs `nowtrend_internal`, bridged in one composite but hard-coded to 0 in another);

UNIT HETEROGENEITY with no declared contract (`gradient_ratio` is a 1-10x multiple, `first_timer_ratio` is a 0-1 fraction, every other component is 0-100, and nothing declares which is which); ENUM OVERLOADING (one `risk_stage` column populated by three engines with three disjoint vocabularies); and UNVALIDATED AI SCALARS (no clamp to [0,100], no enum membership check, partial JSON silently defaulting components to 0).

The structural root is singular: the engine has method docstrings but no declared, machine-checkable FORMAT contract for scoring inputs, and the composite weight vector itself is copied into three implementations that disagree. Until that is fixed, grading and gap analysis can run on silently-missing inputs while appearing complete - so input-layer accuracy is not yet defensible.

The remedy mirrors the date fix exactly: a declared `SCORING_CONTRACT` registry enforced at the write, plus a Scoring Contract Auditor that audits the live data and auto-covers new fields.

1. The Five Questions, Answered

Q1. Clear guidelines for each scoring method? [PARTIAL]

Each component has a prose docstring (e.g. `compute_nowtrendin_score` documents '0-70 volume + 0-30 recency'; the maturity multipliers are documented). But the COMPOSITE weight vector is stated inline in THREE places - `score_topic`, `apply_calibration`, and `calibration_engine.apply` - and they DISAGREE. There is method-level documentation but no single authoritative spec, and no cross-agent contract.

Q2. Clear format requirement per input field? [NO]

Exactly one machine-readable per-field format registry exists (`DATE_SEMANTIC`) and it covers only dates. No declared type / unit / range / key / JSON-shape / enum exists for any other scoring input. The gap between the date contract and every other input is total in kind, not degree.

Q3. Is the data formatted to the requirement? [MOSTLY, with confirmed violations]

Where a format is implied, producers usually conform - but because the requirement is informal, 'conform' is undefined and real violations slip through (`engagement_asymmetry` key drift; AI components with missing keys; positioning collapse). You cannot conform to a contract that is not written down.

Q4. Does the data properly populate (no silent null/0)? [NO - the core failure]

The dominant read idiom across `apply_calibration` (~20 reads), the market engine, and the AI grade is `s.get(key, 0)` or `0` - which makes a MISSING value and a real 0 indistinguishable, and feeds the 0 into the weighted score. Confirmed live: positioning collapses to a flat baseline; AI components default to 0; N-detail columns are absent so the served N breakdown does not match the N that was computed.

Q5. Are all agents/formulas consistent on format? [NO]

Three composite implementations carry disagreeing weight vectors. `nowtrendin_score` vs `nowtrend_internal` key drift is bridged in one path and hard-coded to 0 in another. The `*_detection` / `*_confidence` mode keys are read by the calibrator but never produced by the scorer (masked by a fallback today; a new consumer re-introduces the zero). `risk_stage` is one column overloaded with three enum vocabularies.

2. The Structural Root - One Disease, Six Shapes

Every confirmed issue is the same producer/consumer format mismatch the date bug was. It appears in six recurring shapes:

Pattern	What happens	Confirmed example
Null-coalesce masking	s.get(k,0) or 0 turns MISSING into a real-looking 0 that enters the weighted sum	Positioning Concentration collapses to flat baseline; AI components default to 0
Key drift	Producer writes key A; consumer reads key B -> None -> 0	engagement_asymmetry (scoring path); nowtrendin_score vs nowtrend_internal
Weight-vector drift	One composite formula copied into 3 implementations that disagree	score_topic vs apply_calibration vs calibration_engine.apply
Unit heterogeneity	Mixed 0-100 / 0-1 / ratio with no declared unit per field	gradient_ratio (1-10x), first_timer_ratio (0-1), components (0-100)
Enum overloading	One column carries 2-3 disjoint enum vocabularies	risk_stage: market tier vs risk vocab vs classification
Swallowed / stale	try/except: pass hides a parse fail; derived field goes stale vs its inputs	serve-time calibration swallow; heisenberg_gap staleness

The common thread: **none** of these are caught by any existing check, because the only format contract that is machine-enforced is the date one. A wrong-format scoring input produces no error and no log - it produces a plausible-looking score on missing data. That is precisely the condition the founder identified: accurate grading and gap analysis are not possible when inputs are silently misread.

3. Confirmed Issues (adversarially verified)

Of 42 candidates across the five lenses, 15 high-severity were probed against live data; 9 survived adversarial verification as real. Listed worst-first:

#	Sev	Issue	Fix (summary)
1	medium	AI-grade scores are NOT on the same scale as engine scores: ai_grade.py uses the OLD pre-renormalization component weights (with N still present), then hardcodes the N input to 0.0 - leaving 0.12 of "dead" detection weight and 0.10 of dead	In transfer/ai_grade.py, make the AI-grade path use the SAME renormalized 6-component weights as the engine (gravitational_anomaly_detector.py:3919-3936), so AI scores land on the engine scale. Prefe
2	high	AI-grade detection/confidence (and the derived heisenberg_gap) are computed with LEGACY 7-component weights that include N (det 0.12, conf 0.10) while N is hardcoded to 0.0 and NEVER renormalized out - so every AI-graded topic is silently p	Renormalize the six AI-grade components to 1.0 by dropping N entirely, matching the engine. Either (a) define 6-component weight dicts in ai_grade.py identical to the engine's serve weights (det: G0.3
3	high	CONFIRMED silent-misread: the Positioning Concentration market component silently collapses to a flat, non-informative baseline value (z=0.0 / score 30.0) for 100% of risk topics because its positioning inputs (FINRA short_interest change_p	Two coupled fixes. (1) STOP masking missing-as-zero on the positioning path: in market_signal_engine.assemble_market_components, distinguish "input absent" from "input genuinely 0". Only build positio
4	medium	KEY DRIFT silent misread confirmed on the SCORING path. The producer (gravitational_anomaly_detector.py:4044) writes engagement-asymmetry evidence under the key "engagement_asymmetry" (int 1/0). The active scoring-time consumer, signal_cali	In signal_calibration_integration.py:1067 read the producer's actual key with the serve alias as fallback: asymmetry = raw_result.get("engagement_asymmetry_detected", raw_result.get("engagement_asymme
5	medium	CONFIRMED silent misread (bounded to display/explainability, NOT the composite score). The N-component "Community demand" detail fields served at /scores/{key} - total_queries_30d, queries_24h, daily_rate_7d - are structurally always 0. The	Add the three missing columns and write them, so the served N-detail reflects the same numbers N was computed from. (1) Schema: add to velocity_scores (723-766) `nowtrending_queries_30d INTEGER DEFAULT
6	MEDIUM	CONFIRMED (real, latent): The single `risk_scores.risk_stage` column is populated from TWO producers with DISJOINT enum vocabularies, and the downstream consumer is built for only ONE of them. At transfer/financial_risk_gradient.py:2434 the	Stop overloading one column with two enum vocabularies. Preferred: at financial_risk_gradient.py:2434, do NOT fall back across vocabularies - when `mkt` is present write the market tier, when it is ab
7	medium	risk_scores.risk_stage is one column populated from TWO disjoint enum vocabularies depending on a branch at write time: financial_risk_gradient.py:2434 writes `_stage = mkt["tier"]` if mkt else score["risk_stage"]`. When a market gradient ex	Make risk_stage a single declared vocabulary, or namespace the two. Concretely: (1) In financial_risk_gradient.py at line 2434, do not store raw `mkt["tier"]` into the risk_stage column under a differ
8	medium	Real but LATENT silent-downgrade hole, not an active live misread. In transfer/gravitational_anomaly_detector.py the serve-time calibration transform _calibrate_score_fields (lines 7977-8067) wraps each of the three score-mutating steps - a	Make the swallow observable and fail-visible without removing the legitimate module-absent fallback (the _CAL_AVAILABLE / _AI_INTEL_AVAILABLE / _CORRECTIONS_AVAILABLE guards already handle a missing m
9	medium	Partial verification of the candidate. The CITED FIELD and SYMPTOM are real (AI-proposed components silently default to 0.0 and still produce a "real" proposed detection/confidence score on the user-facing Grade tool), but the candidate's S	In grade_topic (transfer/ai_grade.py ~line 451, before building comps), validate that the required component keys are PRESENT and numeric in the score dict. Treat a missing/non-numeric component as a

4. The Fix - a Scoring Data Contract (proposed canon)

The date model already proved the pattern works: a declared registry (DATE_SEMANTIC), a central normalizer (to_iso_date), a write-time gate that QUARANTINES rather than guesses (gate_date), and an auditor that checks the live data (Agent 16). Generalize it to every scoring input.

The rule (hard, enforceable)

- ONE canonical key per scoring field - no aliases. Every producer and every consumer use it. The three composite weight vectors collapse to ONE definition, imported everywhere (kills weight-vector drift).
- A SCORING_CONTRACT registry declares, per field: canonical key, type (scalar/json/bool/enum), UNIT and range (0-100 / 0-1 / ratio / count), allowed enum set with casing, JSON shape, required-non-null flag, and the producing function. This is the machine-readable spec Q2 is missing.
- Writes pass through a normalizer that validates type/unit/range/enum. A value it cannot conform is QUARANTINED to a review queue - never silently defaulted. MISSING is represented as None and is DISTINGUISHABLE from a real 0; scoring formulas decide explicitly how to treat missing (skip + renormalize, or flag insufficient-data) - never let .get(k,0) or 0 smuggle it in as zero.
- AI-returned scalars are clamped to their declared range and AI enums are membership-checked before they enter any score (closes the unvalidated-AI-scalar hole and the AI-grade N-weight circularity).
- The contract is AUDITED on the data, not assumed - by the Scoring Contract Auditor below.

Example registry entries

Field	Type	Unit / range	Enum / shape	Required	Producer
gradient_strength	scalar	0-100	-	yes	compute_gradient_strength
gradient_ratio	scalar	ratio 1-10x	-	yes	compute_gradient_strength
first_timer_ratio	scalar	0-1 fraction	-	no	compute_dark_matter
engagement_asymmetry	bool	0/1	-	no	score_topic
nowtrendin_score	scalar	0-100	-	yes	compute_nowtrendin_score
signal_stage	enum	-	BREAKOUT/STRONG/EMERGING/MARGINAL/MONITORING	yes	stage_of
risk_stage	enum	-	ELEVATED/ACTIVE/BUILDING/ROUTINE/DORMANT	yes	compute_market_signal
platforms_active	json	list[str]	[...]	yes	score_topic

5. The Guardrail - Scoring Contract Auditor (Agent 17)

Read-only, in /monitor run_all, endpoint /monitor/scoringcontract - the exact pattern that worked for the Canonical Date Auditor (Agent 16), now for all scoring fields. It audits the DATA, so a violation is caught no matter which code path produced it, and new fields are auto-covered.

What it checks

- For every field in SCORING_CONTRACT, across all live rows: TYPE conformance (scalar/json/bool/enum), UNIT/RANGE conformance (a 0-1 field never exceeds 1; a 0-100 field never exceeds 100), ENUM membership (with casing), and NON-NULL where the field is required.
- REQUIRED-but-NULL / 0 rate per field: a scoring input that is null or exactly 0 on an implausible share of rows is the silent-misread signature (this is what would have caught Positioning Concentration collapsing to a flat baseline).
- KEY-DRIFT / weight-vector divergence: assert the single composite weight vector is the one used on every path, and that producer keys equal consumer keys (no alias bridging).
- DISCOVERY: any scoring-shaped column NOT in the registry is flagged 'classify it' (warn) - so a new source or new component is covered automatically, exactly like the date auditor's *_date discovery.

Success vs error (so the audit gives genuine feedback)

State	Output
SUCCESS	status=ok; 0 type/unit/enum violations; 0 required-but-null; 0 unregistered scoring columns; the single weight vector matches on every path.
ERROR (critical)	Names field + table + row count + examples: e.g. 'positioning_concentration: 84/88 rows = flat baseline 30.0 (missing-as-zero)'; 'ai_grade detection uses 7-comp weights incl N'.
ERROR (warn)	An unregistered scoring-shaped column (classify: add to SCORING_CONTRACT), or a producer/consumer key mismatch that is currently masked by a fallback.

6. Remediation Roadmap

Quick wins - mechanical, low risk, ship now

- AI-Grade weights: drop N and renormalize to the engine's six-component vector - fixes BOTH the scale mismatch and the N-into-Gradient circularity (highest-value single fix; HIGH severity).
- engagement_asymmetry: read the producer's actual key (with the serve alias as fallback) on the scoring path in apply_calibration.
- AI scalars: clamp every AI-returned number to its declared range and membership-check AI enums before they enter a score; reject/flag partial JSON instead of defaulting components to 0.
- N-detail: add the three missing community-demand columns (or stop serving a stale N breakdown) so the served detail matches the N that was computed.
- Make the serve-time calibration swallow observable: log on failure instead of silent except: pass (keep the legitimate module-absent fallback, but never hide a real parse error).

Structural - gated by backtest-before-ship (these change WHICH data enters a score)

- Build the SCORING_CONTRACT registry + a write-time normalizer; replace s.get(k,0) or 0 on scoring paths with contract-validated reads that distinguish MISSING from real-zero.
- Consolidate the three composite weight vectors into one definition imported everywhere.
- Split risk_stage into one declared enum vocabulary (or namespace the two/three); reconcile tier / risk_stage / classification to one surfaced source of truth.
- Resolve the market detection/confidence single-column-two-formulas split and the WATCHLIST_TICKERS exact-string match that silently drops all company financials on a name drift.
- Ship the Scoring Contract Auditor (Agent 17) and add it to /monitor run_all; wire its findings into the nightly audit alongside the Canonical Date Auditor.

Integrity note. Most structural items change which inputs reach a score, so they MOVE scores and must pass the project's backtest-before-ship rule - validated against the held-out Wikipedia/GDELT referee, never auto-applied. The quick wins are mechanical and do not require a calibration backtest, except the AI-grade weight fix, which should be spot-checked because it changes graded values (correctly).